DNS-Based Traffic Steering for IPv6-Enabled Edge Microservices

Extended Project Description

Joar Heimonen
contact@joar.me

June 7, 2026

Contents

1 Introduction

The modern cloud computing stack relies heavily on Layer 7 proxies and service meshes to manage traffic between services however this comes at a cost. These layer 7 proxies introduce single points of failure in otherwise distributed systems, whilst adding unnecessary complexity and reducing the effectiveness of layer 3 routing. Service meshes add sidecar containers to every pod. This overhead scales with the amount of microservices you have.

This architectural pattern did not emerge by design. IPv4 exhaustion forced the development of technologies like NAT (Network Address Translation) and shared address spaces. With the introduction of NAT end-to-end accessibility was broken making proxies a structural necessity.

With a rapid increase in IPv6 adoption it is time to leave the technical limitations of the past in the past and build our services with IPv6s large address space in mind. Under RIPE-738 [1], a Local internet registry receives a minimum allocation size of CIDR /32 up to /29 without needing to justify it. A /29 allocation alone contains around 147 billion times more addresses than the whole IPv4 address space.

This thesis investigates whether DNS-based traffic steering, combined with direct IPv6 addressing of microservices can serve as a viable alternative to traditional layer 7 ingress proxies and service meshes. Rather than routing traffic through a centralized proxy each pod is assigned a globally routable address and made directly reachable by clients. Traffic steering decisions are delegated to a purpose built DNS server that dynamically manages record based on pod health and metrics.

The remainder of this document surveys the state of the art in traffic steering and proxy scalability, identifies the gap that motivates this research and outlines the proposed approach, experimental methodology and work plan.

2 State of the Art in Traffic Steering and Proxy Scalability

2.1 The End-to-End Principle and the Role of NAT

Saltzer, Reed and Clark established in 1984 that application specific functions should be implemented at the endpoints of a communication system rather than within the network itself [2]. A network that simply moves packets without enforcing control points is inherently more resilient. There are no forced bottlenecks, no single points of failure introduced by the infrastructure itself. Packets are free to be routed as the network sees fit.

The widespread adoption of NAT under IPv4 fundamentally broke this principle. By placing address translation logic inside the network, NAT made end-to-end addressability impossible. Applications could no longer communicate directly over the internet. They required proxies, STUN servers and other middleboxes to facilitate communication compensating for the loss of globally unique addressing.

IPv6 restores the conditions under which end-to-end addressability principle can be applied. With a globally unique address assigned to every endpoint there is no longer a structural requirement for address translation or centralized proxies. The network can return to its intended role, moving packets between endpoints that are directly reachable by design.

2.2 Layer 7 Proxies and Ingress Controllers

A reverse proxy sits between the client and the backend services as a mandatory intermediary. This intermediary is responsible for TLS termination, load balancing, health checking and routing. These are functions that every service in the system depends on, making the proxy both a critical component and an inherent bottleneck.

HAProxy, NGINX, Envoy and Traefik represent the dominant L7 proxy implementations. Despite their differences in design and feature set, they share a common architectural assumption. All traffic must pass through a userspace process that terminates connections, inspects application-layer content, and forwards requests to backend services. Wang, Shou, Qian and Liu quantify the cost of this assumption, sidecar-based proxies reduce throughput by up to 55.9% and more than double latency even under light load, with only 20% of that overhead attributable to actual load balancing logic [3].

The centralization is the problem. All traffic passes through one process that becomes a bottleneck and a single point of failure in an otherwise distributed system. The more services present the worse it gets.

2.3 Service Mesh Architectures

Service meshes are a backbone that is supposed to allow services to arbitrarily talk to each other. Most modern service meshes can be seen as consisting of two planes. A data plane and a control plane. The data plane consists of sidecar proxies injected into every pod, whilst the control plane distributes configurations, certificates and routing rules to the sidecar proxies whilst maintaining a global graph of the mesh.

When pod A calls pod B the traffic flows from pod A to pod A's sidecar proxy, through the network and then to pod B's sidecar proxy. The sidecar proxy then relays the call to the pod. On top of this the control plane is constantly rotating certificates, changing routing rules, updating configurations and managing service discoverability. It becomes its own distributed system just for passing messages between nodes of another distributed system.

Li, Lu, Lin, Wu, Zhang and Yan measure the performance of this architecture directly. Deploying a service mesh more than doubles communication latency, decreases throughput by over 60%, and consumes over 70% of CPU resources [4]. The large overhead is not incidental, it is a consequence of routing every inter-service call through multiple user-space processes, each crossing the kernel boundary multiple times per request. This results in an alarming amount of context switches for a system that is supposed to be efficient.

2.4 Kernel and Hardware Acceleration

A recent development in the field is the moving of L7 logic into the kernel. Rather than eliminating the proxy, these approaches attempt to reduce its overhead by removing the large overhead caused by crossing the kernel user-space boundary multiple times per request.

Wang, Shou, Qian and Liu propose XLB, an in kernel L7 load balancer implemented using eBPF (Extended Berkeley Packet Filtering). By intercepting traffic in the socket layer rather than routing it through a separate user-space process XLB reduces the cross process communication overhead that makes traditional sidecar proxies expensive. The result is up to 1.5x higher

throughput and 60% lower latency [3]. However, this approach is currently not viable as it requires re-implementing user-space protocols in the kernel. XLB only supports unencrypted HTTP 1.1 and support for newer protocols would require the implementation of a full in kernel layer 7 network stack, including support for websockets, webtransports, HTTP 3.0 (quic support) and TLS support.

Miziński and Przyłucki confirm the broader trend, finding that Cilium's eBPF based networking reduces CPU and memory consumption significantly under load compared to traditional iptables based approaches. however, this comes at a slight cost to raw throughput [5].

Li, Lu, Lin, Wu, Zhang and Yan propose FlatProxy, which migrates the service mesh proxy onto a Data Processing Unit (DPU). A DPU is a dedicated hardware component, in FlatProxys case it is a Field Programmable Gate Array (FPGA). The FlatProxy reduces overhead by running the proxy on the DPU rather than the CPU. However, to avoid implementing L7 protocols on an FPGA they have devised a simpler solution. The DPU routes traffic that it has not seen before to a user space process on FlatProxy's CPU. The user-space process then creates routing rules that are applied to all future proxies by the DPU. FlatProxy eliminates resource contention between proxy and application, achieving a 90% reduction in latency and 4x improvement in throughput compared to Envoy [4]. However, DPU-based approaches introduce significant hardware requirements that make them unsuitable for commodity or edge deployments.

Both hardware accelerated proxies and in kernel proxies share a fundamental assumption with the systems they are trying to improve upon: traffic must still pass through a dedicated proxy. The overhead is reduced but, the centralization remains the same.

2.5 DNS-Based Traffic Steering

The Domain Name System (DNS) is part of the backbone of the internet. It is what allows our services to remain static as the address space evolves. Rather than routing traffic through a proxy DNS-based traffic steering directs clients to the appropriate backend before the connection is even established. The steering decision happens at or before resolution time instead of at request time.

Hawley demonstrated DNS-based geographical traffic steering in 2009 with GeoDNS, deployed at kernel.org to distribute Linux kernel downloads across mirrors in multiple countries [6]. The approach is protocol agnostic and completely transparent to the servers and clients. Notably, DNS-based steering was not considered a fringe idea at the time. Wikipedia, Mozilla and the Apache foundation all operated GeoDNS infrastructure at the time. The concept predates all modern cloud paradigms.

DNS is hierarchical. Responsibility for a zone is delegated from parent to child through a chain of authority. At the top of this chain sits the root zone, delegating to top-level domains, which delegate to authoritative nameservers for individual domains. Each zone has a Start Of Authority (SOA) record that identifies the primary nameserver responsible for that zone and defines the default TTL value for its records.

The canonical limitation of DNS-based steering is TTL caching. Moura, Heidemann and Schmidt demonstrate that recursive resolvers frequently cache records beyond their stated TTL, and that the effective cache lifetime of a record can far exceed what the operator intended [7]. Shaikh, Tewari and Agrawal show that reducing TTL to improve steering responsiveness increases nameserver query load by orders of magnitude [8]. This creates a fundamental tension between steering

freshness and infrastructure cost. Chatterjee, Tari and Zomaya propose adapting TTL values dynamically to the workload rather than fixing them, reducing client perceived latency while limiting the query load cost [9].

This may be mitigated through proper delegation of authority. By delegating a dedicated subzone to its own authoritative nameserver and ensuring that recursive resolvers are only served NS records for that zone, clients are forced to contact the authoritative nameserver directly. The stub resolver on the client machine, upon receiving an NS record, bypasses the recursive resolver's cache entirely and queries the authoritative server. This means steering decisions are always fresh, regardless of what recursive resolvers choose to cache, because the client never receives a cacheable A or AAAA record from the recursive resolver. However, modern DNS stub resolvers and recursive servers often only loosely adhere to DNS standards, making the practical effects of fringe delegation mechanisms difficult to predict.

The same delegation can serve multiple NS records, so a resolver that cannot reach one authoritative nameserver retries another, giving the steering infrastructure the same decentralized failover.

Returning multiple records gives clients alternatives, and modern clients act on them through Happy Eyeballs (RFC 8305), which races connection attempts across the returned addresses and proceeds with whichever responds first [10]. A dead backend is simply the address that loses the race, so the client fails over to a working one at connection time, with no central health checker. Where a proxy detects failure through periodic health checks, a Happy Eyeballs client detects it immediately as part of connecting.

to our knowledge no one has applied dns-based steering at the per-pod level in a microservice environment.

2.6 Media over QUIC and the Fanout Proxy

Not every proxy performs the per request steering this thesis seeks to eliminate. Media over QUIC (MoQ) is an up and coming protocol for efficient low latency media streaming, currently progressing trough the IETF standards process [11]. It is built around a network of relays. A publisher emmits a stream of objects to a relay, the relay forwards those objects onto its subscribers. The subscribers may themselves be further relays. In this way a single source is fanned out to a large audience without the publisher needing to reach each subscriber directly.

What distinguishes MoQ from the proxies surveyed above is that it performs no per request processing on the data it carries. The specifications requires the relay to treat the object payload as opaque, and forbids it from combining, splitting or otherwise modifying payloads when forwarding [11]. A subscription is established once, and from that point the relay simply forwards objects along an already decided path. The relay does not inspect content to make routing decisions per message. It is a pipe instead of a router.

A relay and a steering proxy are therefore doing fundamentally different jobs. A steering proxy makes a routing decision on every request, work the endpoints could in principle perform. A relay performs a fanout. The delivery of one stream to many recipients, which a single publisher cannot achieve alone. One is a convenience place in the hot path. The other is a function that can only exist in the network. MoQ shows that a relay stripped of per request processing remains a complete and capable architecture, suggesting that the per request decisions, not the relay itself is the part that can be removed.

2.7 Distributed systems at the Edge

Historically cloud native systems have been designed around centralized datacenters where services operate within a relatively stable and well connected environment. The emergence of 5G has renewed interest in Multi-access Edge Computing (MEC), where computational resources are deployed closer to users. By deploying smaller datacenters alongside 5G access infrastructure, computation can be performed closer to the point where traffic enters the network, reducing latency and lowering strain on the wider internet.

Unlike traditional cloud deployments, a MEC environment might consist of hundreds of geographically distributed deployments. Architectures that rely on centralized traffic steering must therefore operate over a much larger number of sites. As a system grows, the cost and complexity

of maintaining centralized coordination grows. Jeffery, Howard and Mortier demonstrate this directly in Kubernetes, showing that etcd's consistent datastore degrades in write latency and throughput as the cluster grows, becoming a bottleneck that limits scalability at the edge [12]. They argue that the bottleneck stems from etcd's requirement for consistency: every node must agree to a write before it is confirmed. Relaxing this requirement, so that nodes converge over time instead, removes the bottleneck. The tradeoff is immediate consistency for availability and scalability [12].

In contrast, architectures that rely primarily on local decision making can expand more organically, allowing new deployment sites to be added without proportionally increasing the burden on centralized infrastructure.

Similar principles can be observed in large-scale infrastructure that has evolved over time, such as electrical power networks. Rather than being designed around a fixed topology, power networks have gradually expanded through the addition of new power plants, substations and transmission lines. The architecture permits incremental growth, allowing the system to adapt to changing demand without requiring fundamental redesign. This illustrates an important property of large-scale infrastructure. Systems that are built around local connectivity and loosely coupled components can often scale more organically than systems that depend on centralized coordination.

3 Literature Search Methodology

The literature search was conducted using Google Scholar and Google Scholar Labs. Search terms were initially generated with the assistance of a large language model based on the research topic, and subsequently refined through iterative searching. The following keyword queries were used:

- *DNS TTL load balancing consistency tradeoff*
- *IPv6 end-to-end addressing NAT elimination*
- *eBPF Kubernetes networking performance*
- *service mesh overhead scalability microservices*
- *DNS-based server selection load balancing*

Papers were selected based on relevance to the research problem, publication venue quality, and citation count. The search was supplemented by backward snowballing from the reference lists of key papers, through which several foundational works were identified. Papers published in predatory or low-quality journals were excluded.

The resulting set of nine papers is summarized in Table 1.

#	Authors	Year	Key Claim
1	Wang, Shou, Qian, Liu	2026	In-kernel eBPF L7 load balancer, 1.5x throughput over Istio [3]
2	Li, Lu, Lin, Wu, Zhang, Yan	2023	DPU-centric service mesh, 90% latency reduction vs Envoy [4]
3	Hawley	2009	GeoDNS geographic traffic steering at Internet scale [6]
4	Jeffery, Howard, Mortier	2021	etcd strong consistency bottleneck in edge Kubernetes [12]
5	Miziński, Przyłucki	2025	Cilium eBPF vs iptables: lower CPU/memory under load [5]
6	Moura, Heidemann, Schmidt	2019	DNS TTL tradeoffs between caching efficiency and routing flexibility [7]
7	Shaikh, Tewari, Agrawal	2001	Reducing DNS TTL increases nameserver load by orders of magnitude [8]
8	Chatterjee, Tari, Zomaya	2005	Adaptive TTL approach reduces client-perceived latency by 16% [9]
9	Saltzer, Reed, Clark	1984	End-to-end principle: functions belong at endpoints, not in the network [2]

Table 1: Papers included in the literature review

4 Problem Description

After reviewing a wide spread of articles a pattern becomes clear: The state of the art is focused on proxy efficiency rather than proxy elimination. Every approach surveyed from userspace proxies to service meshes to hardware and kernel based solutions, reduces the cost of the central proxy without questioning whether it needs to exist at all. XLB moves the load balancer into the kernel to avoid the userspace boundary [3]. FlatProxy moves it onto dedicated hardware to avoid competing with the application for CPU [4]. Both make the proxy faster. Neither removes it. The traffic still passes trough a dedicated node that every request depends on.

This shared assumption is not incidental, it is inherited. As established in Section 2.1, the proxy became a structural necessity when NAT broke end-to-end addressability under IPv4. With every endpoint forced behind a translation layer, a middlebox was the only place certain functions could live. The efficiency focused work improves this middlebox without revisiting the constraint that created it. IPv6 removes the constraint. With a globally routable address on every pod, there is no longer a structural reason for the steering decision to happen within the hot path.

This raises the question this thesis investigates. Can the central proxy be eliminated entirely, rather than merely being optimized? An ingress proxy and a service mesh sidecar, viewed as steering mechanisms do the same job, they intercept a request and direct it to a backend. The defining property of this job is that it happens per request. DNS based steering cannot do this. It steers per resolution, handing the clients a set of addresses before the connection is established.

The core claim of this thesis is that per request steering is not necessary for the workloads that dominate distributed systems, and that steering at or before resolution time rather than per request is sufficient to replace the centralized proxy.

This is not a free substitution. Steering per resolution rather than per request means the system cannot redirect a connection once it has been established. A client pinned to a backend stays there for the life of the connection. For short lived or frequently re-resolving clients this costs nothing. For long lived multiplexed connections it can be a real limitation, and identifying where this limitation becomes the limiting factor is part of what this research sets out to measure. This thesis is a first step towards the broader claim that per request steering is unnecessary. It does not prove universally, but it tests whether a proxy less DNS steered architecture holds up against the approaches that keep the central proxy.

5 Proposed Approach and Methodology

This thesis proposes that the central proxy can be removed entirely rather than optimized, and aims to evaluate that proposal experimentally. Each pod is assigned its own globally routable IPv6 address and made directly reachable by clients. In place of a Layer 7 ingress proxy, a purpose-built DNS server performs steering by managing the records returned for a service, selecting backends according to pod health and metrics. Steering decisions are therefore made at or before resolution time, and traffic flows directly from client to pod without passing through an intermediary process.

The core of the work is the design and implementation of a minimal DNS server that functions as a Kubernetes ingress controller. Rather than routing traffic it dynamically manages the address records for each service based on live pod metrics, returning a spread of healthy backends and withdrawing records for pods that are unhealthy or overloaded. Failover is handled by the mechanisms described in Section 2.5, multiple records combined with Happy Eyeballs at the connection layer, and multiple NS records at the authoritative layer. No central health checker or request-path proxy is required.

To evaluate this architecture it is compared against baseline paradigms drawn from the systematization above, configured according to best practice. The evaluation uses dummy microservices and a client simulator deployed in a Kubernetes cluster. The number of pods is held constant while the number of simulated clients is varied, isolating the effect of client load on each paradigm. The metrics collected are round trip time, throughput, CPU utilisation, and failover behaviour following simulated pod and node crashes. Comparing these across the DNS-based approach and the Layer 7 baselines allows the central question to be answered: whether steering at or before resolution can match the request-path proxies it seeks to replace, and where, if anywhere, the per resolution model breaks down.

6 Conclusion

References

[1] *IPv6 Address Allocation and Assignment Policy*, https://www.ripe.net/publications/docs/ripe-738/. Accessed: May 21, 2026.

[2] J. H. Saltzer, D. P. Reed, and D. D. Clark, "End-to-end arguments in system design," *ACM Transactions on Computer Systems*, vol. 2, no. 4, pp. 277–288, Nov. 1984, ISSN: 0734-2071, 1557-7333. DOI: 10.1145/357401.357402 Accessed: May 21, 2026.

[3] Y. Wang, C. Shou, J. Qian, and G. Liu, *XLB: A High Performance Layer-7 Load Balancer for Microservices using eBPF-based In-kernel Interposition*, 2026. DOI: 10.48550/ARXIV.2602.09473 Accessed: May 21, 2026.

[4] M. Li, W. Lu, H. Lin, J. Wu, Y. Zhang, and G. Yan, "FlatProxy: A DPU-centric Service Mesh Architecture for Hyperscale Cloud-native Application," vol. 14, no. 8, 2023.

[5] K. Miziński and S. Przyłucki, "The impact of using eBPF technology on the performance of networking solutions in a Kubernetes cluster," *Journal of Computer Sciences Institute*, vol. 35, pp. 150–158, Jun. 2025, ISSN: 2544-0764. DOI: 10.35784/jcsi.7110 Accessed: May 21, 2026.

[6] J. Hawley, "GeoDNS—Geographically-aware, protocol-agnostic load balancing at the DNS level,"

[7] G. C. M. Moura, J. Heidemann, R. d. O. Schmidt, and W. Hardaker, "Cache Me If You Can: Effects of DNS Time-to-Live," in *Proceedings of the Internet Measurement Conference*, ser. IMC '19, New York, NY, USA: Association for Computing Machinery, Oct. 2019, pp. 101–115, ISBN: 978-1-4503-6948-0. DOI: 10.1145/3355369.3355568 Accessed: May 21, 2026.

[8] A. Shaikh, R. Tewari, and M. Agrawal, "On the effectiveness of DNS-based server selection," in *Proceedings IEEE INFOCOM 2001. Conference on Computer Communications. Twentieth Annual Joint Conference of the IEEE Computer and Communications Society (Cat. No.01CH37213)*, vol. 3, Apr. 2001, 1801–1810 vol.3. DOI: 10.1109/INFCOM.2001.916678 Accessed: May 21, 2026.

[9] D. Chatterjee, Z. Tari, and A. Zomaya, "A task-based adaptive TTL approach for Web server load balancing," in *10th IEEE Symposium on Computers and Communications (ISCC'05)*, Jun. 2005, pp. 877–884. DOI: 10.1109/ISCC.2005.19 Accessed: May 21, 2026.

[10] D. Schinazi and T. Pauly, "Happy Eyeballs Version 2: Better Connectivity Using Concurrency," Internet Engineering Task Force, Request for Comments RFC 8305, Dec. 2017. DOI: 10.17487/RFC8305 Accessed: Jun. 7, 2026.

[11] S. Nandakumar, V. Vasiliev, I. Swett, and A. Frindell, "Media over QUIC Transport," Internet Engineering Task Force, Internet Draft draft-ietf-moq-transport-18, May 2026. Accessed: Jun. 7, 2026.

[12] A. Jeffery, H. Howard, and R. Mortier, "Rearchitecting Kubernetes for the Edge," in *Proceedings of the 4th International Workshop on Edge Systems, Analytics and Networking*, ser. EdgeSys '21, New York, NY, USA: Association for Computing Machinery, Apr. 2021, pp. 7–12, ISBN: 978-1-4503-8291-5. DOI: 10.1145/3434770.3459730 Accessed: May 21, 2026.